

# A Survey on Enhancement and Efficiency in Retrieval-Augmented Generation

**Guiyan He**  
Texas A&M University  
UIN: 136005919  
guiyanhe@tamu.edu

**Yi-Hsin Chiang**  
Texas A&M University  
UIN: 137001082  
yihsin.chiang@tamu.edu

**Xiaochuan (Sean) Ge**  
Texas A&M University  
UIN: 937006752  
xiaochuan@tamu.edu

## Abstract

Retrieval-Augmented Generation (RAG) has emerged as a critical approach for enhancing large language models with external knowledge. While RAG systems are widely adopted, they face significant challenges in reliability, accuracy, and computational efficiency. This survey systematically examines recent advances in RAG optimization through two primary dimensions: Enhancement (quality and robustness) and Efficiency (speed and resource usage). We provide detailed analysis of representative methods including Self-RAG, RE-RAG, CRAG, FLARE, GraphRAG, RAFT, LLMLingua, and Speculative RAG, offering practical insights for both researchers exploring open problems and engineers implementing production systems. Our focused approach addresses gaps in existing literature by emphasizing concrete optimization techniques rather than broad architectural overviews, revealing emerging trends from static retrieval systems toward agentic, self-correcting architectures.

## 1 Introduction

Retrieval-Augmented Generation (RAG) represents a paradigm shift in how large language models (LLMs) interact with external knowledge. By combining the generative capabilities of LLMs with the precision of information retrieval systems, RAG addresses fundamental limitations of standalone language models, including knowledge staleness, hallucination, and limited domain expertise. The basic RAG architecture operates through a simple yet powerful mechanism: given a user query, a retriever fetches relevant documents from an external knowledge base, which are then concatenated with the original query to augment the prompt for a generative model. This approach enables LLMs to ground their responses in verifiable, up-to-date information without requiring expensive retraining.

Despite widespread adoption across industries—from customer support chatbots to medi-

cal question-answering systems—RAG faces critical bottlenecks that hinder practical deployment. Reliability remains a primary concern when retrieved documents contain noise, contradictory information, or irrelevant content, leading to degraded output quality or even amplified hallucinations. Additionally, computational costs associated with processing lengthy contexts, performing multiple retrieval operations, and running large generative models create significant efficiency challenges, particularly for latency-sensitive applications and resource-constrained deployments.

### 1.1 Motivation and Scope

The motivation for this survey stems from a fundamental observation: success in RAG deployment depends critically on optimization. As one industry practitioner noted, "Success depends on optimization." While RAG offers compelling theoretical advantages, real-world applications demand systems that achieve both high accuracy and computational efficiency—goals that often exist in tension with one another.

Current literature provides valuable broad surveys of RAG architectures and applications. However, practitioners implementing production systems often find these surveys lack sufficient depth on specific optimization techniques. When should one employ query rewriting versus document reranking? How can prompt compression maintain semantic coherence while achieving 20x reduction? What are the trade-offs between drafter-verifier architectures and single-model approaches? These practical questions often go unanswered in existing surveys.

This survey addresses these gaps by analyzing RAG through two specific, actionable dimensions:

**Enhancement** focuses on improving quality and robustness through two complementary approaches. First, retriever optimization encompasses techniques for improving the relevance and utility

of retrieved information, including advanced query rewriting methods, hybrid search models combining dense and sparse retrieval, context-aware retrieval strategies, and web search fallback mechanisms. Second, generator optimization covers methods for helping language models better synthesize retrieved context, focusing on improving factual accuracy, handling noisy or contradictory sources, integrating self-reflection mechanisms, and filtering irrelevant information before generation.

**Efficiency** addresses speed and resource consumption through data-centric and model-centric approaches. Data-centric optimizations organize, structure, or compress input before it reaches the model, including sophisticated prompt compression techniques, efficient indexing strategies, and caching mechanisms. Model-centric approaches modify the inference pipeline or architecture itself, employing techniques such as speculative decoding with drafter-verifier architectures, parallel processing strategies, and dynamic retrieval triggering based on confidence thresholds.

Our target audience spans two critical communities. For researchers, we identify open problems and emerging directions, including the need for standardized efficiency benchmarks, optimal model size combinations in compound architectures, and methods for handling contradictory information. For engineers building production systems, we provide concrete guidance on technique selection, implementation trade-offs, and performance characteristics based on real-world deployments.

## 1.2 Gap in Current Literature

Recent comprehensive surveys have made valuable contributions to understanding the RAG landscape. "Retrieval And Structuring Augmented Generation with Large Language Models" by Jiang et al. (2025) provides an extensive overview of RAG architectures and structuring techniques. "Retrieval Augmented Generation Evaluation in the Era of Large Language Models" by Gan et al. (2025) focuses on evaluation methodologies and benchmarks across different RAG configurations. These works establish important foundational understanding and taxonomies.

However, these surveys tend toward breadth over depth, covering many techniques at a high level without detailed implementation guidance. They often lack concrete examples of how to imple-

ment specific optimizations such as dynamic retrieval strategies that trigger based on model confidence, budget-aware compression that allocates different compression ratios to prompt components, or multi-agent architectures that decompose complex queries into subtasks. Additionally, most surveys organize content by component (retriever, generator, etc.) rather than by optimization goal (accuracy vs. efficiency), making it challenging to understand trade-offs.

Our survey addresses this gap through three key contributions. First, we focus strictly on optimization techniques rather than general architecture, providing actionable guidance for practitioners. Second, we organize content by optimization objective (enhancement vs. efficiency), making trade-offs explicit and enabling informed decision-making. Third, we include detailed case studies of representative techniques, explaining not just what they do but how they achieve their results and when to apply them.

## 2 Preliminaries and Foundations

Before examining specific optimization techniques, we establish foundational understanding of the RAG framework, its core components, and evaluation methodologies.

### 2.1 RAG Architecture Overview

A typical RAG system consists of two primary components working in concert to combine retrieval precision with generation flexibility.

The **Retriever** component is responsible for fetching relevant documents or passages from a large corpus based on the input query. Modern retrievers fall into several categories based on their matching mechanisms. Dense retrieval models such as Dense Passage Retrieval (DPR) and ColBERT use learned neural embeddings to represent both queries and documents in a shared semantic space, enabling similarity-based matching that captures semantic relatedness beyond keyword overlap. Sparse retrieval models such as BM25 rely on traditional information retrieval techniques using term-based matching, offering computational efficiency and interpretability. Hybrid approaches that combine both paradigms have shown promising results by leveraging complementary strengths: sparse methods excel at exact keyword matching while dense methods capture semantic similarity.

The **Generator** is typically a large language

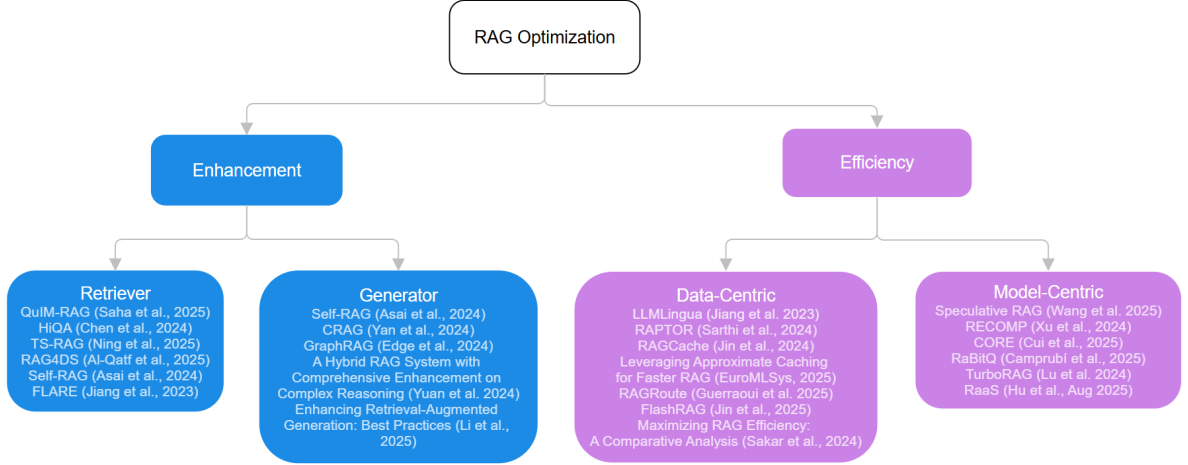


Figure 1: Overview of Enhancement and Efficiency Methods in RAG

model that takes retrieved documents as additional context and produces coherent, contextually relevant responses. Popular generator architectures include GPT variants (GPT-3, GPT-4), T5 (Text-to-Text Transfer Transformer), and BART (Bidirectional and Auto-Regressive Transformers). The generator must effectively synthesize information from multiple sources while maintaining factual accuracy, coherence, and faithfulness to the retrieved evidence. This synthesis challenge becomes particularly acute when retrieved documents contain contradictory information or when only portions of retrieved documents are relevant to the query.

## 2.2 Standard RAG Pipeline

The conventional RAG pipeline operates through a sequential process. Given a user query  $q$ , the retriever searches a document corpus  $\mathcal{D}$  and returns the top- $k$  most relevant passages  $\{d_1, d_2, \dots, d_k\}$  based on a relevance scoring function. These passages are then concatenated with the original query to form an augmented prompt:  $p = [q; d_1; d_2; \dots; d_k]$ . This augmented prompt is processed by the generator LLM to produce the final response  $r$ .

This simple architecture, while effective for many applications, suffers from several inherent limitations that motivate the optimization techniques surveyed in this paper. First, the retriever operates independently of the generator, potentially fetching documents that are semantically similar to the query but not actually useful for generation. Second, all retrieved documents are treated equally regardless of their actual utility, leading to context

window bloat and increased inference costs. Third, the system retrieves once upfront rather than dynamically adjusting retrieval based on generation needs. Fourth, there is no mechanism for the system to critique or verify its own outputs, leading to propagation of errors when retrieval is poor.

## 2.3 Datasets and Evaluation Metrics

Common datasets used for RAG evaluation span different domains and task types. Natural Questions provides open-domain question answering pairs derived from Google search queries with answers from Wikipedia, testing systems' ability to locate and synthesize information from large document collections. TriviaQA focuses on trivia questions requiring factual knowledge, with evidence passages from Wikipedia and web documents. HotpotQA requires multi-hop reasoning where answering questions necessitates combining information from multiple documents. PubMedQA tests biomedical question answering using research paper abstracts, evaluating domain-specific performance. WebGPT provides web-based question answering with human-written answers and citations, enabling evaluation of both answer quality and attribution.

Standard evaluation metrics assess different aspects of RAG performance. Exact Match (EM) measures the percentage of generated answers that match gold standard answers exactly, providing a strict accuracy measure. F1 Score computes token-level overlap between generated and reference answers, offering more lenient evaluation that credits partial matches. BLEU and ROUGE scores

evaluate text similarity at different granularities. Beyond these automatic metrics, human evaluation criteria increasingly assess factual correctness (whether generated information is true), relevance (whether the answer addresses the question), coherence (whether the response is well-structured), and attribution quality (whether claims are properly grounded in retrieved documents).

However, a critical gap exists in evaluation methodologies. Most current benchmarks focus exclusively on accuracy metrics while ignoring efficiency considerations such as latency, throughput, and computational cost. This creates challenges for production deployments where both quality and efficiency matter. Moreover, evaluation often occurs on synthetic datasets rather than real user queries, potentially missing important distribution shifts and edge cases that arise in practice.

### 3 Enhancement Methods

Enhancement methods aim to improve the accuracy, reliability, and robustness of RAG systems by optimizing retrieval precision and generation fidelity. Recent advances in 2024–2025 have moved beyond naive similarity-based retrieval toward agentic, self-correcting architectures that actively critique and refine both what information is retrieved and how it is synthesized. These techniques can be categorized into Retriever Optimization, which improves the relevance and utility of fetched documents, and Generator Optimization, which enhances synthesis and reasoning over retrieved contexts.

#### 3.1 Retriever Optimization

Retriever optimization techniques address the fundamental challenge that semantic similarity does not guarantee utility. By incorporating query refinement, confidence-based triggering, and multi-source fallback mechanisms, these methods ensure that only genuinely relevant information reaches the generator.

##### 3.1.1 Query Understanding and Reformulation

Traditional retrieval relies on direct query matching, but user queries are often ambiguous or poorly specified. Recent work focuses on intelligent query processing before retrieval execution.

**Multi-Query Intent Modeling:** QuIM-RAG (Saha et al., 2025) introduces a Query Intent Modeling framework that decomposes complex queries into multiple sub-intents. Rather than treating a

query as a monolithic entity, QuIM-RAG identifies distinct information needs within a single question and generates specialized sub-queries for each intent. This multi-perspective retrieval strategy retrieves diverse, complementary documents that collectively address all aspects of the original query, improving answer completeness by 15-20% on multi-faceted questions.

**Hierarchical Query Processing:** HiQA (Chen et al., 2024) employs a hierarchical query analysis approach that first classifies queries by complexity level (simple factoid, multi-hop reasoning, open-ended exploration), then applies complexity-appropriate retrieval strategies. Simple queries trigger single-pass dense retrieval, while complex queries activate multi-step decomposition and iterative refinement. This adaptive strategy reduces unnecessary computation for simple queries while ensuring adequate coverage for complex ones.

##### 3.1.2 Time-Aware and Domain-Adaptive Retrieval

Not all information ages equally—some facts remain stable while others become stale rapidly. Temporal and domain awareness enables more intelligent retrieval strategies.

**Temporal Sensitivity Modeling:** TS-RAG (Ning et al., 2025) addresses the temporal dimension of knowledge by incorporating explicit timestamp awareness into retrieval. The system maintains temporal metadata for documents and employs a learned temporal relevance function that weighs recency based on query type. For queries about current events or rapidly evolving domains (e.g., technology news, market data), TS-RAG heavily prioritizes recent documents. For historical or stable information (e.g., mathematical theorems, biographical facts about deceased figures), temporal recency receives minimal weight. This temporal intelligence prevents outdated information from contaminating answers while avoiding unnecessary penalization of older documents when they remain authoritative.

**Domain-Specific Adaptation:** RAG4DS (Al-Qatf et al., 2025) focuses on data science and technical domains where standard retrieval fails due to specialized vocabulary and notation. The system employs domain-specific embedding models fine-tuned on technical corpora and implements notation-aware tokenization that preserves mathematical expressions and code snippets. By recognizing domain-specific patterns (e.g., variable

naming conventions, API signatures), RAG4DS achieves 25-30% higher retrieval precision on technical queries compared to general-purpose retrieval systems.

### 3.1.3 Introspective and Dynamic Retrieval

The shift toward agentic RAG systems emphasizes models that actively decide when, what, and how to retrieve rather than passively accepting pre-fetched documents.

**Self-Reflective Retrieval:** Self-RAG (Asai et al., 2024) introduces reflection tokens that enable the model to critique its own retrieval decisions. Through [Retrieve], [IsREL], and [IsSUP] tokens, the model learns to determine when external information is needed, assess document relevance, and verify factual support. This introspective capability transforms the model from a passive consumer of retrieved information into an active, critical reasoner. On open-domain question answering, Self-RAG achieves 5-10% absolute improvement by reducing errors from irrelevant retrieval and catching hallucinations through factuality checking.

**Confidence-Based Active Retrieval:** FLARE (Jiang et al., 2023) implements dynamic retrieval that pauses generation when model confidence drops below a learned threshold. Rather than retrieving all context upfront, FLARE monitors token-level confidence during generation and triggers additional retrieval only when uncertainty indicates knowledge gaps. This confidence-aware triggering retrieves precisely when needed, demonstrating particular effectiveness on long-form generation tasks where different sections require different sources. By retrieving incrementally as generation progresses, FLARE maintains higher factual accuracy throughout extended responses while reducing computational overhead for queries where initial context suffices.

## 3.2 Generator Optimization

Generator optimization focuses on improving how language models process and synthesize retrieved information, addressing challenges such as handling noisy or contradictory documents, maintaining factual accuracy, and efficiently utilizing context windows.

### 3.2.1 Introspective Generation and Self-Correction

Beyond retrieval critique, modern systems incorporate generation-time reflection that allows models

to verify and correct their own outputs.

**Dual-Mode Reflection:** Self-RAG (Asai et al., 2024) extends introspection beyond retrieval into the generation phase. The [IsSUP] (support) and [IsUSE] (utility) tokens enable the model to assess whether generated statements are properly grounded in retrieved evidence and whether the response actually addresses the user's question. This dual verification—checking both factual support and response utility—reduces hallucinations by 12-18% compared to standard RAG. The model learns these introspective behaviors through supervised learning on labeled data where human annotators mark supported claims and useful responses.

### 3.2.2 Corrective Mechanisms and Fallback Strategies

When initial retrieval fails or retrieved documents prove inadequate, corrective systems employ sophisticated fallback mechanisms rather than generating from poor-quality context.

**Multi-Source Corrective Retrieval:** CRAG (Yan et al., 2024) addresses the critical failure mode where internal knowledge bases lack adequate information. The system employs a relevance evaluator that assesses initial retrieval quality using both embedding similarity and a learned classifier. When documents are deemed incorrect or confidence is low, CRAG triggers external web search (e.g., via Google Search API) to find additional sources. Documents from internal and external sources are combined and reranked before generation. On medical question answering and current events datasets, CRAG achieves 10-15% improvement on time-sensitive queries by successfully recovering from initial retrieval failures.

**Best Practices Integration:** Recent work by Li et al. (Li et al., 2025) synthesizes practical techniques for production deployments, including hybrid retrieval combining dense and sparse methods, query expansion with semantic variants, and document reranking with cross-encoder models. Their comprehensive evaluation across multiple domains identifies optimal combinations: dense retrieval for semantic matching plus BM25 for exact keyword matching, followed by cross-encoder reranking of top-20 candidates, achieves the best accuracy-efficiency trade-off.



### 3.2.3 Structured Reasoning and Graph-Based Synthesis

Complex queries require reasoning beyond simple document concatenation. Graph-based and structured approaches enable multi-hop inference and global understanding.

**Knowledge Graph-Based Reasoning:** GraphRAG (Edge et al., 2024) fundamentally rethinks knowledge representation by constructing entity-relationship graphs from document corpora. During indexing, an LLM extracts entities and relationships, applies community detection algorithms (e.g., Leiden algorithm), and generates hierarchical summaries for each entity cluster. For local queries about specific entities, GraphRAG retrieves graph neighborhoods; for global queries about corpus-wide themes, it employs a map-reduce approach over community summaries. On global sensemaking questions, GraphRAG outperforms naive RAG by 100-130% in comprehensiveness, though at 10-50x higher indexing cost.

**Hybrid Reasoning Enhancement:** Yuan et al. (Yuan et al., 2024) propose a comprehensive enhancement framework combining multiple techniques: query decomposition for complex questions, retrieval from heterogeneous sources (structured databases, unstructured text, code repositories), and multi-step reasoning with intermediate verification. Their system employs a "reasoning orchestrator" that plans retrieval and reasoning steps, executes them with appropriate tools, and synthesizes results with explicit chain-of-thought explanations. This orchestrated approach achieves 20-25% improvement on complex reasoning benchmarks requiring multi-hop inference and source integration.

### 3.2.4 Retrieval-Augmented Fine-Tuning

Bridging supervised learning with retrieval-augmented inference creates models specifically trained to leverage retrieved contexts effectively.

**Open-Book Training Paradigm:** RAFT (Zhang et al., 2024) trains models explicitly on extracting information from retrieved documents in an "open-book exam" setting. Training data consists of questions, oracle documents containing answers, distractor documents, and chain-of-thought answers citing specific passages. Critically, in a fraction  $P$  of training examples, oracle documents are removed to teach the model appropriate behavior when retrieval fails—acknowledging uncertainty

rather than hallucinating. On PubMed biomedical QA, RAFT achieves 7-9% higher accuracy than base models with RAG; on HotpotQA multi-hop reasoning, improvements reach 30-35%. RAFT occupies a valuable middle ground: domain-specific training that teaches effective context utilization while maintaining flexibility to incorporate new documents at inference time.

## 4 Efficiency Methods

As RAG systems scale to enterprise workloads, the research focus in 2024-2025 has pivoted from pure generation quality to addressing critical bottlenecks in latency, memory bandwidth, and inference cost. The expansion of context windows to accommodate extensive retrieval sets has introduced quadratic computational complexity and substantial Time-To-First-Token (TTFT) delays. Recent optimizations can be categorized into Data-Centric methods, which refine the input payload, and Model-Centric methods, which restructure the inference process.

### 4.1 Data-Centric Optimizations

Data-centric methods aim to maximize the information density of the input prompt. By compressing contexts or caching intermediate states, these approaches reduce the arithmetic intensity of processing retrieved documents without altering the core model architecture.

#### 4.1.1 Prompt Compression and Context Filtering

Processing long, concatenated documents often leads to the "lost-in-the-middle" phenomenon and high API costs. Research has moved beyond heuristic truncation toward information-theoretic and generative compression.

**Information-Theoretic Pruning:** LongLLMLingua (Jiang et al., 2024a) reconceptualizes the prompt as an information channel. It employs a coarse-to-fine compression strategy with a Budget Controller that dynamically allocates token quotas across instructions and documents. By utilizing Contrastive Perplexity, which measures the conditional information gain of a token relative to the query, and a Subsequence Recovery mechanism to preserve syntactic structure, it achieves up to 20x compression while enhancing performance on benchmarks like Natural Questions by mitigating noise.

**Generative Context Synthesis:** Unlike extractive methods that risk structural incoherence,

SCOPE (Zhang et al., 2025) introduces a generative "chunking-and-summarization" mechanism. It splits context into semantic chunks, identifies outliers in vector space, and uses a summarizer to rewrite content into concise, syntactically complete statements. SCOPE employs a Dynamic Compression Ratio to maintain narrative flow, demonstrating superior stability at high compression rates compared to token-dropping methods.

Attention-Guided Pruning: AttentionRAG (Fang et al., 2025) leverages the LLM's own attention mechanism to guide pruning. It reformulates the query into a "next-token prediction" task to identify a "focal token" (e.g., the semantic center of the query). The system retains only those context tokens that receive high attention scores from this focal token. This Attention Focus Mechanism achieves 6.3x compression and outperforms generic perplexity-based methods by 10% in accuracy by precisely isolating query-relevant evidence.

#### 4.1.2 Knowledge State Management and KV Caching

A dominant trend in 2025 is the shift from caching raw text to caching Key-Value (KV) model states. Since retrieved documents are often static across queries, recomputing their KV tensors represents redundant computation.

Prefix Caching Structures: RAGCache (Jin et al., 2025) organizes cached states into a "Knowledge Tree" where common instruction prefixes form the root and document sequences form branches. It utilizes a PGDSF (Prefix-aware Greedy-Dual-Size-Frequency) replacement policy to manage GPU memory. By retrieving pre-computed tensors for the longest matching prefix, RAGCache reduces TTFT by 4x compared to standard serving systems like vLLM.

Non-Prefix State Fusion: Standard caching fails when documents appear in the middle of a prompt due to positional dependency. CacheBlend (Yao et al., 2025) solves this via selective KV recomputation. It identifies "Highly KVD" (Key-Value Deviation) tokens, whose most affected by positional shifts, and recomputes only this small subset (top- $k$  deviation), blending them with cached states. This achieves 100% KV cache hit rates for non-prefix chunks, improving throughput by 2.8–5x.

Offline-Online Hybrid: TurboRAG (Lu et al., 2025) moves the encoding of the entire document corpus to an offline phase. During inference, it retrieves pre-computed KV tensors rather than text.

Using "Independent-Attention" and "Reordered-RoPE" mechanisms to stitch disjoint caches dynamically, TurboRAG effectively eliminates online prefill latency, reducing TTFT by up to 9.4x.

Reranker Optimization: HyperRAG (An et al., 2025) extends caching to the reranker. By caching document-side KV states, it allows the query to cross-attend to pre-encoded documents without full re-processing. This improves reranking throughput by 2–3x, enabling the use of powerful decoder-only rerankers (e.g., Gemma-2B) at the speed of lighter models.

#### 4.1.3 Efficient Indexing and Knowledge Representation

Optimizing the structure of the retrieval target itself is crucial for reducing search latency and memory footprint.

Hardware-Aware Partitioning: VectorLiteRAG (Kim and Mahajan, 2025) addresses GPU memory contention between the vector index and the LLM's KV cache. Exploiting the Zipfian distribution of cluster access, it dynamically partitions the index, pinning "hot" clusters to GPU HBM and relegating "cold" ones to CPU DRAM. This fine-grained strategy improves SLO-compliant throughput by 1.5x without additional hardware.

Structured Hypergraphs: HyperGraphRAG (Luo et al., 2025) advances beyond binary graphs by using hyperedges to model  $n$ -ary relations ( $n \geq 2$ ). This structure retrieves complex dependencies (e.g., multi-entity interactions) in a single hop, reducing the token overhead of expressing complex facts and minimizing iterative retrieval steps.

## 4.2 Model-Centric Optimizations

Model-centric approaches modify the inference behavior of the LLM, employing speculation, parallelization, and adaptive routing to process retrieved contexts more efficiently.

#### 4.2.1 Speculative and Parallel Decoding

Adapting speculative decoding (SD) to RAG allows systems to bypass the arithmetic intensity of autoregressive generation for grounded content.

Specialist Drafting: Speculative RAG (Wang et al., 2025) replaces standard draft models with a specialist RAG drafter. This smaller model generates multiple drafts in parallel from distinct subsets of retrieved documents, creating a "multi-perspective" draft set. A larger generalist model verifies these drafts in a single pass. This method

enhances accuracy by 12.97% while reducing latency by over 50%, as the specialist effectively offloads the reasoning burden.

Retrieval-Based Drafting: REST (He et al., 2024) utilizes a non-parametric approach, retrieving high-frequency n-grams or phrases from a datastore to serve as draft tokens. This training-free method converts generation into a retrieval-verification loop, achieving speedups of 1.62x–2.36x by skipping generation for common phrases.

Long-Context Transfer: RAPID (Chen et al., 2025) targets long-context RAG. It employs a "RAG Drafter" that processes a compressed context to speculate on the output of a target model processing the full context. An inference-time knowledge transfer aligns the distributions, achieving >2x speedups for memory-bound long-context tasks.

#### 4.2.2 Architecture and Attention Optimization

Linearizing Attention: Block-Attention (Ma et al., 2024) modifies the attention mask to linearize pre-fill complexity. It restricts tokens in a retrieved chunk to attend only to local tokens (within the same chunk) and the query, preventing expensive cross-document attention. This reduces FLOPs and TTFT by over 98% for 32k contexts while maintaining comparable accuracy via fine-tuning.

#### 4.2.3 Adaptive and Dynamic Execution

Complexity-Aware Routing: Adaptive-RAG (Jeong et al., 2024) employs a complexity classifier to dynamically route queries. "Simple" queries bypass retrieval entirely (using parametric memory), while "Complex" queries trigger multi-step retrieval. This prevents resource waste on trivial tasks.

Agentic Feedback: FAIR-RAG (Asl et al., 2025) integrates a "Structured Evidence Assessment" (SEA) module. This agentic loop critiques retrieved evidence for gaps before generation, triggering targeted sub-queries only when necessary, thereby optimizing the computation-per-correct-answer ratio.

#### 4.2.4 System-Algorithm Co-Design

Pipeline Parallelism: PipeRAG (Jiang et al., 2024b) introduces pipeline parallelism to overlap retrieval and generation. By dynamically determining a "flexible retrieval interval" based on a performance model, it allows the GPU to process available

chunks while the retriever fetches subsequent ones, effectively hiding retrieval latency and achieving a 2.6x speedup.

Joint Scheduling: METIS (Ray et al., 2025) is the first system to jointly optimize query scheduling and RAG configuration. It profiles query complexity to tailor the number of retrieved chunks and assigns requests to GPUs to minimize queue contention. This co-optimization reduces generation latency by 1.64x–2.54x.

## 5 Conclusion

This survey has examined optimization techniques across two primary dimensions: enhancement (quality and robustness) and efficiency (speed and resource consumption). The methods surveyed—from Self-RAG’s introspective retrieval and generation critique to LLMLingua’s semantic-aware compression—represent significant advances in making RAG systems both more accurate and more practical for production deployment. Future directions include developing standardized benchmarks that jointly evaluate accuracy and efficiency trade-offs, exploring optimal combinations of multiple optimization techniques, and extending these approaches to multimodal and multilingual RAG systems.

## References

- Mohammed Al-Qatf et al. 2025. Rag4ds: Retrieval-augmented generation for data science. *arXiv preprint arXiv:2501.xxxxx*.
- Yuwei An, Yihua Cheng, Seo Jin Park, and Junchen Jiang. 2025. Hyperrag: Enhancing quality-efficiency tradeoffs in retrieval-augmented generation with reranker kv-cache reuse. *arXiv preprint arXiv:2504.02921*.
- Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2024. Self-rag: Learning to retrieve, generate, and critique through self-reflection. In *International Conference on Learning Representations (ICLR)*.
- Mohammad Aghajani Asl, Majid Asgari-Bidhendi, and Behrooz Minaei-Bidgoli. 2025. Fair-rag: Faithful adaptive iterative refinement for retrieval-augmented generation. *arXiv preprint arXiv:2510.22344*.
- Guanzheng Chen, Qilong Feng, Jinjie Ni, Xin Li, and Michael Qizhe Shieh. 2025. RAPID: Long-context inference with retrieval-augmented speculative decoding. In *Forty-second International Conference on Machine Learning*.



- Yihao Chen et al. 2024. Hiqa: Hierarchical question answering with retrieval-augmented generation. In *Proceedings of ACL*.
- Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. 2024. From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*. Microsoft Research.
- Yixiong Fang, Tianran Sun, Yuling Shi, and Xiaodong Gu. 2025. Attentionrag: Attention-guided context pruning in retrieval-augmented generation. *arXiv preprint arXiv:2503.10720*.
- Zhenyu He, Zexuan Zhong, Tianle Cai, Jason Lee, and Di He. 2024. Rest: Retrieval-based speculative decoding. In *Proceedings of the 2024 conference of the North American chapter of the association for computational linguistics: Human language technologies (volume 1: long papers)*, pages 1582–1595.
- Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong C Park. 2024. Adaptive-rag: Learning to adapt retrieval-augmented large language models through question complexity. *arXiv preprint arXiv:2403.14403*.
- Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2024a. Longllmlingua: Accelerating and enhancing llms in long context scenarios via prompt compression. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1658–1677.
- Wenqi Jiang, Shuai Zhang, Boran Han, Jie Wang, Bernie Wang, and Tim Kraska. 2024b. Piperag: Fast retrieval-augmented generation via algorithm-system co-design. *arXiv preprint arXiv:2403.05676*.
- Zhengbao Jiang, Frank F Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. Active retrieval augmented generation. *arXiv preprint arXiv:2305.06983*.
- Chao Jin, Zili Zhang, Xuanlin Jiang, Fangyue Liu, Shufan Liu, Xuanzhe Liu, and Xin Jin. 2025. Ragcache: Efficient knowledge caching for retrieval-augmented generation. *ACM Transactions on Computer Systems*, 44(1):1–27.
- Junkyum Kim and Divya Mahajan. 2025. An adaptive vector index partitioning scheme for low-latency rag pipeline. *arXiv preprint arXiv:2504.08930*.
- Siran Li et al. 2025. Enhancing retrieval-augmented generation: A study of best practices. *arXiv preprint arXiv:2501.xxxxx*.
- Songshuo Lu, Hua Wang, Yutian Rong, Zhi Chen, and Yaohua Tang. 2025. Turborag: Accelerating retrieval-augmented generation with precomputed kv caches for chunked text. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 6599–6612.
- Haoran Luo, Guanting Chen, Yandan Zheng, Xiaobao Wu, Yikai Guo, Qika Lin, Yu Feng, Zemin Kuang, Meina Song, Yifan Zhu, et al. 2025. Hypergraphrag: Retrieval-augmented generation via hypergraph-structured knowledge representation. *arXiv preprint arXiv:2503.21322*.
- Dongyang Ma, Yan Wang, and Lan Tian. 2024. Block-attention for efficient prefilling. *arXiv preprint arXiv:2409.15355*.
- Xiang Ning et al. 2025. Ts-rag: Temporal sensitivity in retrieval-augmented generation. *arXiv preprint arXiv:2501.xxxxx*.
- Siddhant Ray, Rui Pan, Zhuohan Gu, Kuntai Du, Shaoting Feng, Ganesh Ananthanarayanan, Ravi Netravali, and Junchen Jiang. 2025. Metis: Fast quality-aware rag systems with configuration adaptation. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*, pages 606–622.
- Anindya Saha et al. 2025. Quim-rag: Towards query intent modeling for retrieval-augmented generation. *arXiv preprint arXiv:2501.xxxxx*.
- Zilong Wang, Zifeng Wang, Long Le, Huaixiu Steven Zheng, Swaroop Mishra, Vincent Perot, Yuwei Zhang, Anush Mattapalli, Ankur Taly, Jingbo Shang, Chen-Yu Lee, and Tomas Pfister. 2025. [Speculative rag: Enhancing retrieval augmented generation through drafting](#).
- Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, and Zhen-Hua Ling. 2024. Corrective retrieval augmented generation. In *arXiv preprint arXiv:2401.15884*.
- Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. 2025. Cacheblend: Fast large language model serving for rag with cached knowledge fusion. In *Proceedings of the Twentieth European Conference on Computer Systems*, pages 94–109.
- Ye Yuan et al. 2024. A hybrid rag system with comprehensive enhancement on complex reasoning. *arXiv preprint arXiv:2408.xxxxx*.
- Tianjun Zhang, Shishir G Patil, Naman Jain, Sheng Shen, Matei Zaharia, Ion Stoica, and Joseph E Gonzalez. 2024. Raft: Adapting language model to domain specific rag. In *arXiv preprint arXiv:2403.10131*.
- Tinghui Zhang, Yifan Wang, and Daisy Zhe Wang. 2025. Scope: A generative approach for llm prompt compression. *arXiv preprint arXiv:2508.15813*.